

IMAGE CLASSIFICATION USING CONVOLUTIONAL NEURAL NETWORK (CNN)

Report File

Project Introduction

FashionNet is a deep learning pipeline designed to classify grayscale images of fashion items into ten distinct categories. The project utilizes a 3-block convolutional architecture to extract spatial features from 28×28 pixel images, achieving high classification accuracy.

Core Objectives

- **Data Processing:** Load and normalize the Fashion MNIST dataset.
- **Model Engineering:** Build a CNN using the Keras Sequential API.
- **Performance Evaluation:** Analyze model accuracy, loss, and misclassifications through a confusion matrix.

Dataset Overview

The model is trained on the **Fashion MNIST** dataset, which serves as a modern replacement for the classic MNIST digits dataset.

<i>FEATURES</i>	<i>DETAILS</i>
<i>Total Images</i>	70,000 (60,000 Training / 10,000 Test)
<i>Resolution</i>	28 x 28 pixels
<i>Colour Space</i>	Grayscale (1 channel)
<i>Target Classes</i>	10 clothing categories (e.g., T-shirt, Trouser, Bag)

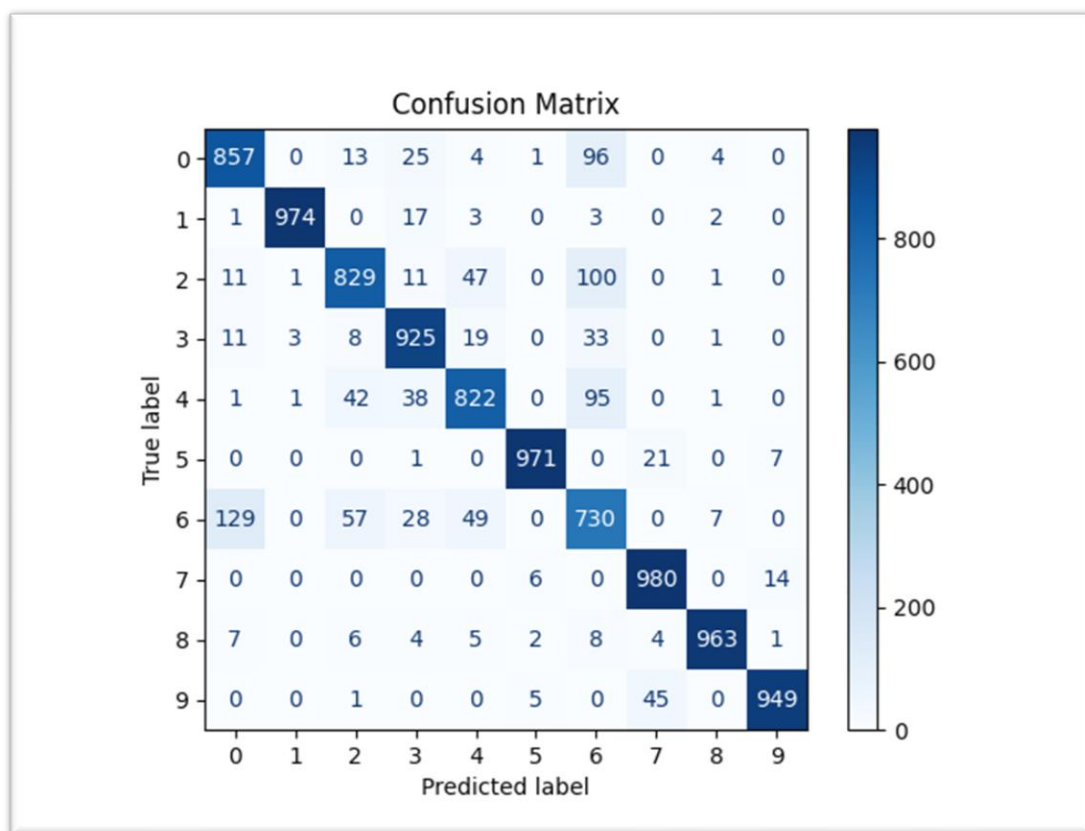
Methodology

The methodology followed in this project includes the following steps:

1. Loading the dataset using TensorFlow.
2. Pre processing the data by normalizing pixel values.
3. Reshaping the dataset for CNN input.
4. Designing the CNN architecture.
5. Training the model using training data.
6. Evaluating the model using test data.
7. Visualizing results using graphs and confusion matrices.

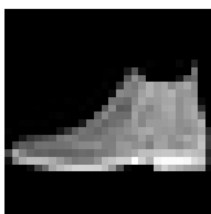
Results





Sample Predictions (Fashion MNIST)

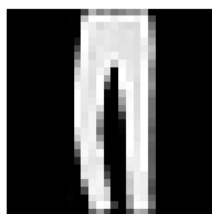
True: Ankle boot
Pred: Ankle boot



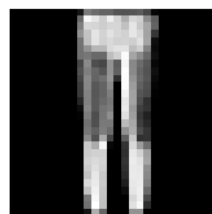
True: Pullover
Pred: Pullover



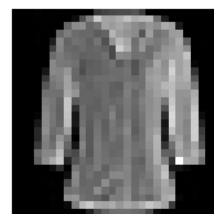
True: Trouser
Pred: Trouser



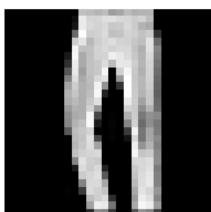
True: Trouser
Pred: Trouser



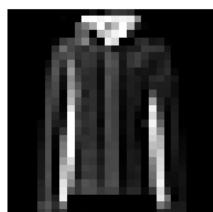
True: Shirt
Pred: Shirt



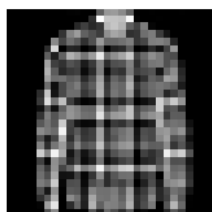
True: Trouser
Pred: Trouser



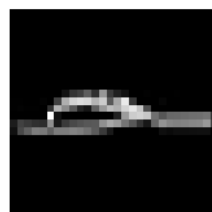
True: Coat
Pred: Coat



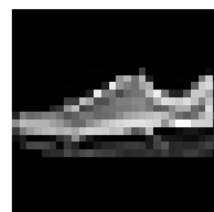
True: Shirt
Pred: Shirt



True: Sandal
Pred: Sandal



True: Sneaker
Pred: Sneaker



System Architecture

The model implements an "improved" CNN design consisting of three convolutional blocks followed by dense layers for classification.

1. **Input Layer:** Accepts 28 x 28 x 1 normalized image data.
2. **Feature Extraction:** * **Block 1:** Conv2D (32 filters, 3 x 3 kernel) + MaxPooling (2 x 2).
 - **Block 2:** Conv2D (64 filters, 3 x 3 kernel) + MaxPooling (2 x 2).
 - **Block 3:** Conv2D (64 filters, 3 x 3 kernel).
3. **Classification Head:**
 - **Flatten Layer:** Converts 2D maps into a 576-dimensional vector.
 - **Dense Layer:** 64 units with ReLU activation.
 - **Output Layer:** 10 units with Softmax activation for class probabilities.

Training and Results

The model was compiled using the Adam optimizer and sparse categorical cross-entropy loss.

Performance Summary:

- **Training Duration:** 5 Epochs with a batch size of 64.
- **Test Accuracy:** 91.3%.
- **Validation Split:** 10% of training data was reserved for real-time performance monitoring.

Top Performing Classes

- **Bags:** 98.2%
- **Trousers:** 97.8%
- **Sandals:** 97.2%

Observation: The model faces its greatest challenge with the "**Shirt**" class (75.1% accuracy), often confusing it with Pullovers or T-shirts due to high visual similarity.

Technical Implementation (Summary)

The implementation was carried out in Python using the following tech stack:

- **Frameworks:** TensorFlow 2.x and Keras.
- **Data Handling:** NumPy and Scikit-learn (for the Confusion Matrix).
- **Visualization:** Matplotlib for learning curves and sample prediction grids.

Conclusion

The **Image Classification** project successfully demonstrates that a relatively lightweight CNN (approximately 93,000 trainable parameters) can effectively classify complex visual patterns in fashion imagery. Future iterations could include Dropout layers to further reduce the slight overfitting observed between training and validation loss.